CS317 Programming Assignment 2 Bonus Problem

Due date: 11:59pm, Apr. 3rd (Wednesday) 2026

Individual Assignment

Implement a BFS-based shortest path algorithm.

Requirements:

Part 1: Algorithm Modification

Modify the breadth-first search (BFS) algorithm, so that it finds the shortest path from a given node *src* to another given node *dst* in a given **undirected connected** graph *G*. Rename the algorithm as *shortestPath.*

ALGORITHM *shortestPath(G, src, dst)*
//Input: Graph $G = (V, E)$, *src*, the source vertex, and *dst*, the destination vertex.
//Output: the shortest path from *src* to *dst*, including all vertices along the path.
…

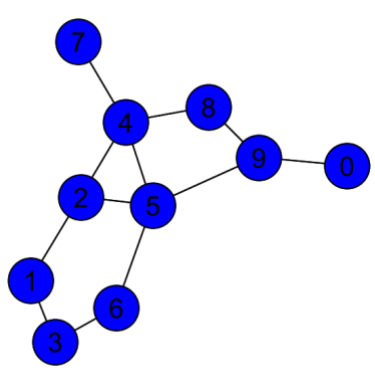
- Following the pseudo code format in the textbook/PPT, define/explain all the symbols/variables used in your algorithm and add comments when necessary.

Part 2 – Algorithm Implementation.

Based on your modified BFS algorithm in part 1, write a BFS-based shortest path program that reads a graph file, prints the graph, and displays the length of the shortest paths from a given vertex to all other vertices. **Your program must meet the following requirements:**

1. Your graph can be represented using either an adjacency matrix or an adjacency list.
2. Your program must include at least the following methods.

1) A method ***readGraph( ),*** which reads graph data from a text file, *graph.txt,* to an adjacency matrix/list. In *graph.txt*, each row represents a link connecting the vertex on the first and second columns. For example, for the following graph,



the *graph.txt* would look like the following:

0, 9
1, 2
1, 3
2, 4
2, 5
3, 6
4, 5
4, 7
4, 8
5, 6
5, 9
8, 9

Note: Since this is an undirected graph, "1 2" implies that "2 1" is also a link. When you create the adjacency matrix, make sure it is a symmetric one. You can assume that the maximum number of vertices in the graph is 50.

2) A method ***printGraph( ),*** which reads the graph data structure and prints the graph in the following format:
v:→ a list of its adjacent vertices

For example, the following is the result of *printGraph( )* for the above graph.

```
Print the graph as follows:
0--> 9
1--> 2  3
2--> 1  4  5
3--> 1  6
4--> 2  5  7  8
5--> 2  4  6  9
6--> 3  5
7--> 4
8--> 4  9
9--> 0  5  8
```

3) Create a method ***shortestPath( )*** based on your algorithm.

Note that here an input, for example, *graph*, does not have to appear as a method argument, i.e., *shortestPath(graph, ...)*, it can appear in a different way, such as *graph.shortestPath(...)*.

Part 3: Sample outputs:

1) The following is an example output based on the previous graph:

```
1. read graph

2. Print graph
0--> 9
1--> 2  3
2--> 1  4  5
3--> 1  6
4--> 2  5  7  8
5--> 2  4  6  9
6--> 3  5
7--> 4
8--> 4  9
9--> 0  5  8

Please enter a starting vertex:
0

The shortest path from 0 to all nodes are:
[0]
[0, 9, 5, 2, 1]
[0, 9, 5, 2]
[0, 9, 5, 6, 3]
[0, 9, 5, 4]
[0, 9, 5]
[0, 9, 5, 6]
[0, 9, 5, 4, 7]
[0, 9, 8]
[0, 9]
```

Hints:

- In the first programming assignment, you have practiced using Java classes that implement a **Queue** ADT. Now it is the time to use that knowledge. Use Java API document and other resources online for details of how to use those classes.

Submission Requirements:

1. **What to submit:** (Submit all required documents to the Canvas.)
   - A .docx or pdf file that contains pseudocode your algorithm. (See part 1).
   - For your code, Zip up your whole project folder and submit it to Canvas. Name the file like **Prog3_YourLastNames.zip**
   - At least three graphs (in .txt format) of at least 10 vertices each. Each graph represents a different test case that you used.
   - Screenshots of the running results of the corresponding test cases. This does not mean that you need only three test cases when testing your program. Try to think about as many different test cases as possible. **Use the RandomGraphVisualization project (Chapter 3: GraphAlgorithms_DFS_WS.zip) to generate random graphs to test your program. Make sure to generate only connected graphs.**

2. Set up a time to give me a demo of your program after your submission.

3. Follow the Java style guide (*JavaStyleGuide.pdf*) in the first programming assignment and comment your code whenever necessary.

4. Your program must be free of compile errors. Programs with compile errors will only earn you a very low grade.

5. Your program must **be your own**. High-level discussions are allowed, but it is not allowed to share code and implementation details. Before starting, please read the "Academic Honesty" section of the syllabus. When you are not sure, please feel free to ask me. Start early and do not wait until the last moment.